

Heart Disease Classification

Decision Tree Model
- Paramjit Kaur



Project Overview

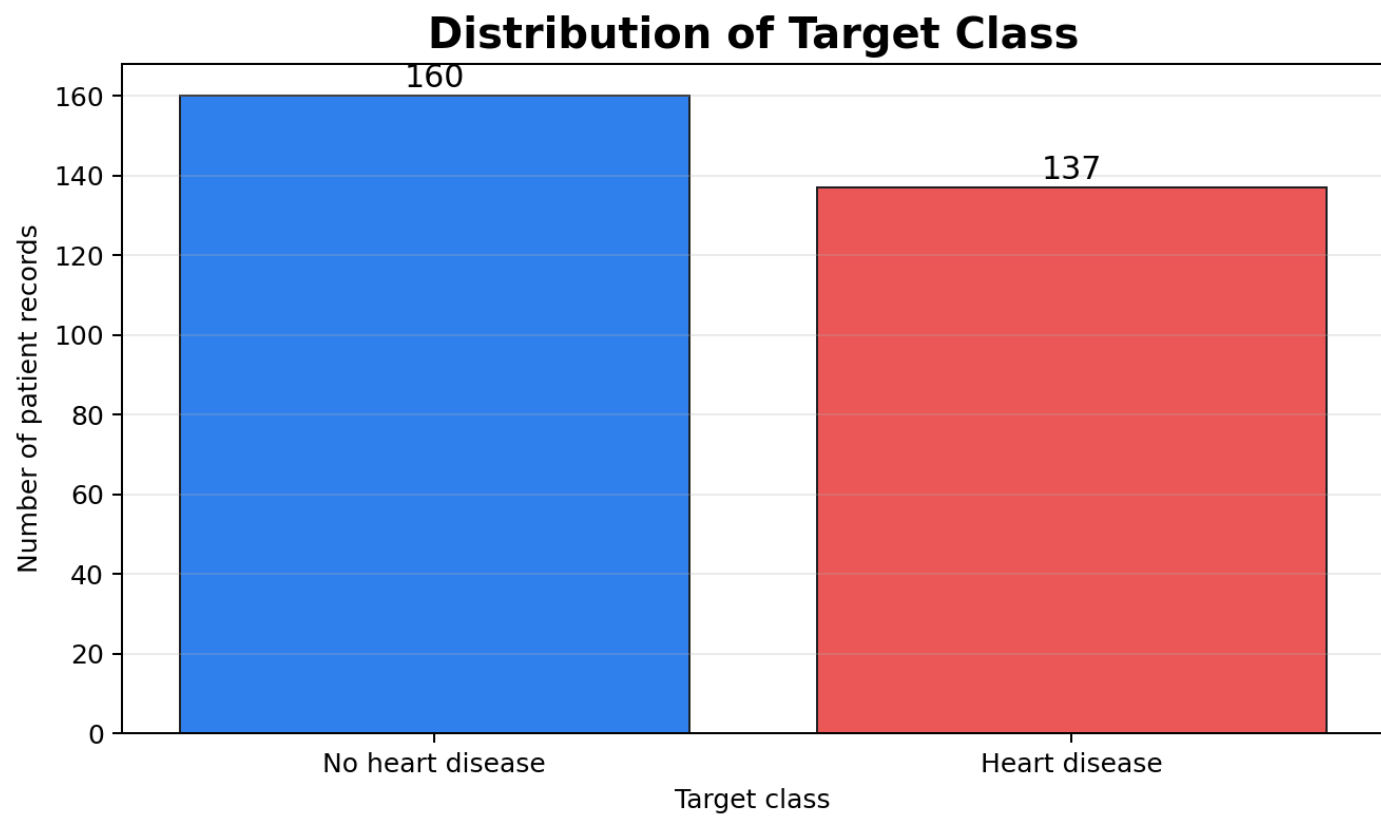
- Dataset: UCI Cleveland Heart Disease dataset, collected from clinical patient records.
- Classification goal: predict whether heart disease is absent or present.
- Model used: one decision tree with `max_depth=3` and `min_samples_leaf=8`.
- Decision Tree was chosen instead of Naive Bayes because medical decision rules are easier to interpret visually.
- Implementation: Python with pandas, scikit-learn, matplotlib, and python-pptx.



Data Description and Source

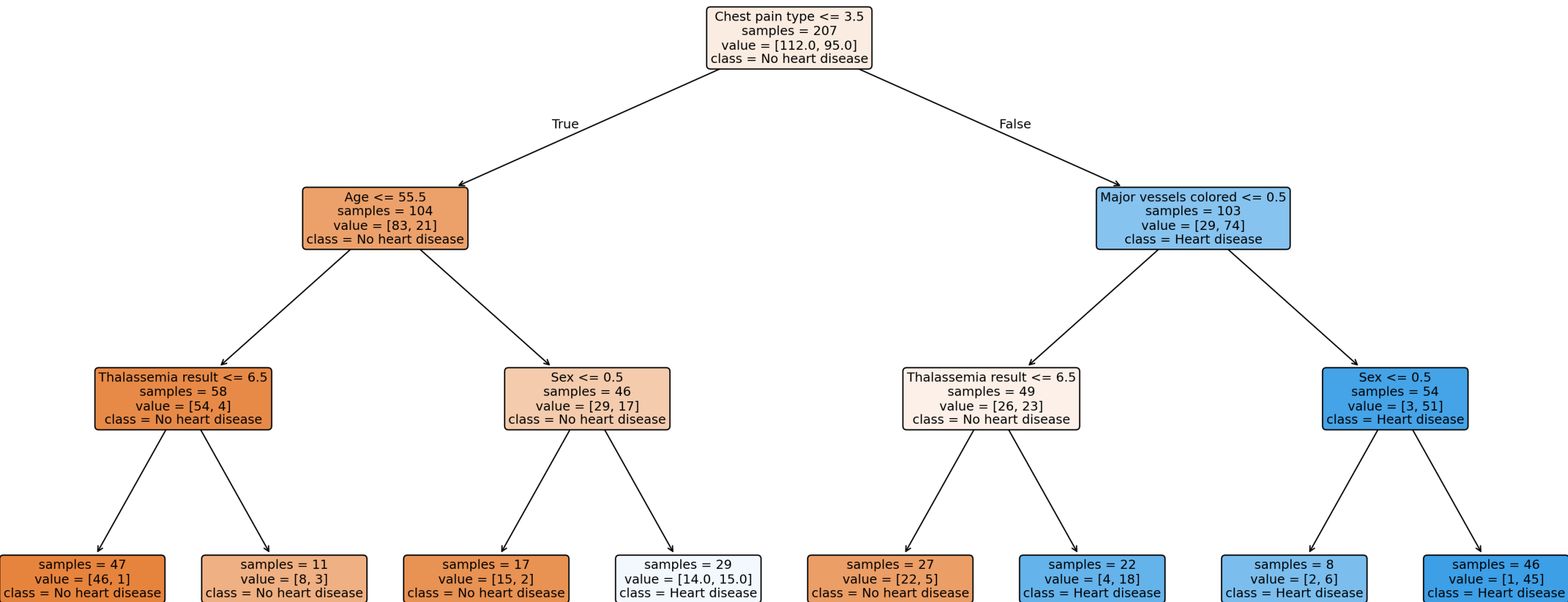
- Usable records after removing missing values: 297 patient records.
- Original attributes include clinical measurements such as age, chest pain type, blood pressure, cholesterol, maximum heart rate, exercise angina, ST depression, vessels colored, and thalassemia result.
- Target field: original values 0-4. This project uses a binary target: 0 = no heart disease, 1 = heart disease present.
- Source: Janosi, Steinbrunn, Pfisterer, and Detrano (1989), Heart Disease dataset, UCI Machine Learning Repository. DOI: 10.24432/C52P4X.
- Dataset URL: <https://archive.ics.uci.edu/dataset/45/heart+disease>

Target Class Distribution



Attributes Used for Training

- Training attributes: Age, Sex, Chest pain type, Resting blood pressure, Serum cholesterol, Fasting blood sugar > 120, Resting ECG result, Maximum heart rate, Exercise-induced angina, ST depression, Slope of ST segment, Major vessels colored, Thalassemia result.
- These attributes were chosen because they are the standard 13 clinical variables used in published experiments with the Cleveland data.
- The original target value was not used as an input feature; it was converted into the binary class label.
- Rows containing missing values in ca or thal were removed so the decision tree could train without imputed values.



Classification Model: Decision Tree

Model Interpretation

- The tree's first split is based on Chest pain type, meaning this attribute gave the strongest initial separation between the two classes.
- The most visible rules combine chest pain type, thalassemia result, maximum heart rate, exercise-induced angina, and ST depression.
- Patients grouped into leaves with abnormal stress-test or chest-pain patterns are more often classified as heart disease present.
- The tree was limited to depth 3 so the resulting model stays readable enough to explain in class.

Evaluation Results

78.9%

Accuracy

78.0%

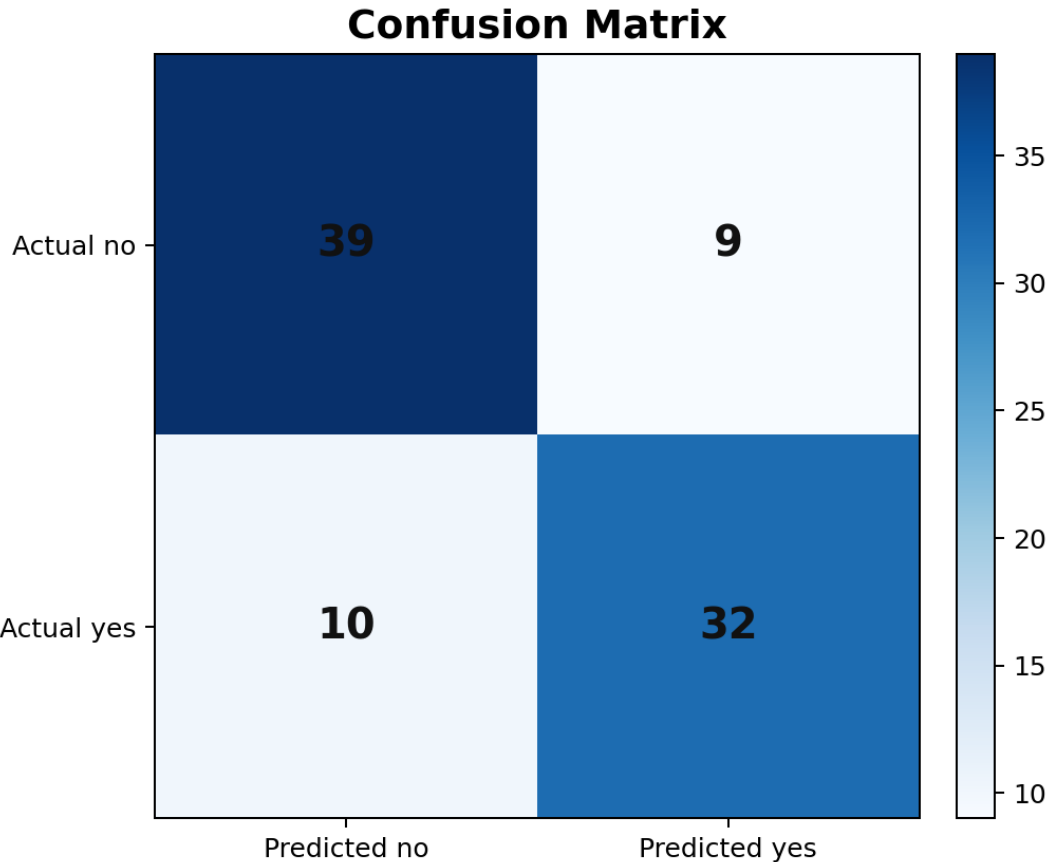
Precision

76.2%

Recall

90

Test records



Evaluation and Summary

- The decision tree achieved 78.9% accuracy, 78.0% precision, and 76.2% recall on the held-out test set.
- The model performed reasonably well for a small and interpretable tree, especially because several clinical attributes are directly related to heart-disease diagnosis.
- Performance may be limited by the small sample size, removed missing-value rows, and the fact that a shallow tree cannot represent every interaction among clinical measurements.
- The shallow tree helped interpretability: the prediction rules are understandable, which is useful in a health-related setting.

How the Work Was Implemented

- The Python script loads the UCI data, assigns column names, removes rows with missing values, and converts the 0-4 target into a binary class.
- The data is split into 70% training and 30% testing with stratification so both class labels appear in both sets.
- A single DecisionTreeClassifier is trained with one setting: max_depth=3 and min_samples_leaf=8.
- The script exports the class distribution chart, tree visualization, confusion matrix, metrics text file, and this slide deck.
- Code file submitted with the slides: classification_project.py.

Implementation Code Excerpt

classification_project.py

```
1 df = pd.read_csv(DATA_FILE, names=FEATURE_NAMES, na_values='?')
2 df = df.dropna().copy()
3 df['heart_disease'] = (df['target_raw'] > 0).astype(int)

5 x_train, x_test, y_train, y_test = train_test_split(
6     x, y, test_size=0.30, random_state=42, stratify=y
7 )

9 model = DecisionTreeClassifier(
10     max_depth=3, min_samples_leaf=8, random_state=42
11 )
12 model.fit(x_train, y_train)
13 y_pred = model.predict(x_test)

15 accuracy = accuracy_score(y_test, y_pred)
16 precision = precision_score(y_test, y_pred)
17 recall = recall_score(y_test, y_pred)
```